

SHIFT CIPHER



VIT[®]
BHOPAL

Encryption & Decryption System

Project Report

Name: Divyansh	Technique: Shift / Caesar Cipher	Input Modes: 3
----------------	----------------------------------	----------------

Based directly on the supplied Python implementation

2. Introduction

This project demonstrates how a classical **Shift Cipher** can be implemented in Python to encrypt and decrypt alphabetic messages.

- **Plaintext:** the original readable message.
- **Cipher key:** the number of positions by which letters are shifted.
- **Encryption:** letters move forward by the key.
- **Ciphertext:** the transformed message.
- **Decryption:** letters move backward by the same key.

The program also supports finding a key from a plaintext/ciphertext pair and provides a menu with three operations.

```
print("MY PROJECT IS IN REFERENCE OF SHIFT CIPHER")  
print("Plaintext: The original, readable message")  
print("Cipher Key: The secret number of shifts")
```

3. Problem Statement

Manual transformation of messages is error-prone. The project provides a simple command-line system that applies a fixed alphabetic shift consistently.

- Convert readable plaintext into ciphertext using a selected key.
- Convert ciphertext back into plaintext using the key.
- Determine the shift key when both plaintext and ciphertext are supplied.
- Handle uppercase and lowercase alphabetic characters.
- Preserve spaces, digits, punctuation, and other non-alphabetic characters.
- Provide an invalid-choice message when the menu selection is outside 1–3.

Example: With key 3, CAT → FDW and FDW → CAT.

4. Functional Requirements

Function	Input	Output / Action
Key Finding	Ciphertext + Plaintext	Calculates shift key from first valid letter pair
Decryption	Key + Ciphertext	Prints decrypted plaintext
Encryption	Key + Plaintext	Prints encrypted ciphertext
Validation	Menu choice	Displays invalid-choice message for other values
Case Handling	Upper/lowercase text	Preserves letter case during encryption/decryption

The program is interactive and receives values through **input()**.

5. Non-functional Requirements

- **Usability:** a simple numbered menu makes the three operations easy to select.

- **Performance:** character-by-character processing is lightweight for normal text inputs.
- **Portability:** uses standard Python features and the built-in **string** module.
- **Maintainability:** the three operations are separated into clear conditional branches.
- **Correctness:** modulo 26 keeps alphabetic shifts inside the A–Z / a–z range.
- **Robustness:** non-letter characters are copied unchanged in encryption/decryption.

Note: this is an educational classical cipher implementation, not a modern secure cryptographic system.

6. System Architecture

The project follows a simple command-line processing architecture.

User
Input & Menu Selection
Python Main Program
Operation Selection: 1 / 2 / 3
Shift Cipher Processing
Console Output

Core logic: read input → select operation → apply shift formula → display result.

7. Design Diagrams

Use Case: User can find a key, decrypt text, or encrypt text.

User	→	Shift Cipher System
	→	1. Find Cipher Key
	→	2. Decrypt Ciphertext
	→	3. Encrypt Plaintext

Workflow: Start → Menu → Choose 1/2/3 → Process → Display result → End

Sequence: User → input() → processing logic → print()

Component: Input layer → Main control → Cipher logic → Output layer

ER Diagram: Not required because the supplied program does not use persistent database storage.

8. Design Decisions & Rationale

- **Shift Cipher:** chosen because its rules are easy to demonstrate and implement.
- **Modulo 26:** allows shifts to wrap from Z back to A.
- **Character-by-character loop:** gives direct control over letters and non-letter characters.
- **Case preservation:** separate handling for A–Z and a–z keeps the original letter case.
- **First valid pair for key finding:** the program calculates the key from the first matching alphabetic positions.
- **Menu-based interface:** one program supports all three required operations.

```
key = (ord(S[i]) - ord(plain_input[i])) % 26
```

9. Implementation Details

Encryption formula

```
cipher = chr((ord(char) - ord('A') + key) % 26 + ord('A'))
```

Decryption formula

```
plain = chr((ord(char) - ord('A') - key) % 26 + ord('A'))
```

Important implementation points

- Uppercase and lowercase letters are processed separately.
- Characters that are not alphabetic are added to the result unchanged.
- The program uses **ord()** and **chr()** for character-code conversion.
- The **string** module is imported, although the core transformations rely mainly on ord/chr.

10. Screenshots / Results

1. Code

```
GNU nano 8.7.1 proj
if a == 'a':
    print("hello we are using shift cipher example today how it works")
    print("1 give cipher text and plain text to find cipher key")
    print("2 give cipher key and cipher text to print plain text")
    print("3 give cipher key and plain text to print cipher text")

    choice = int(input("enter your choice: "))

    if choice == 1:
        S = input("enter a cipher text: ")
        plain_input = input("enter a plain text: ")

        # Convert uppercase characters to lowercase
        word = ""
        for char in S:
            if 'A' <= char <= 'Z':
                word += chr(ord(char) + 32)
            else:
                word += char
        S = word
        wor = ""
        for char in plain_input:
            if 'A' <= char <= 'Z':
                wor += chr(ord(char) + 32)
            else:
                wor += char
        plain_input = wor
        # Calculate cipher key from the first valid letter pair
        key = None
        min_len = min(len(S), len(plain_input))
        for i in range(min_len):
            if S[i].isalpha() and plain_input[i].isalpha():
                key = (ord(S[i]) - ord(plain_input[i])) % 26
                break

        if key is not None:
            print("Your cipher key is:", key)
```

2.

```
GNU nano 8.7.1 proj
if a == 'a':
    print("hello we are using shift cipher example today how it works")
    print("1 give cipher text and plain text to find cipher key")
    print("2 give cipher key and cipher text to print plain text")
    print("3 give cipher key and plain text to print cipher text")

    choice = int(input("enter your choice: "))

    if choice == 1:
        S = input("enter a cipher text: ")
        plain_input = input("enter a plain text: ")

        # Convert uppercase characters to lowercase
        word = ""
        for char in S:
            if 'A' <= char <= 'Z':
                word += chr(ord(char) + 32)
            else:
                word += char
        S = word
        wor = ""
        for char in plain_input:
            if 'A' <= char <= 'Z':
                wor += chr(ord(char) + 32)
            else:
                wor += char
        plain_input = wor
        # Calculate cipher key from the first valid letter pair
        key = None
        min_len = min(len(S), len(plain_input))
        for i in range(min_len):
            if S[i].isalpha() and plain_input[i].isalpha():
                key = (ord(S[i]) - ord(plain_input[i])) % 26
                break

        if key is not None:
            print("Your cipher key is:", key)
```

3--

```
GNU nano 8.7.1      proj
if key is not None:
    print("Your cipher key is:", key)
else:
    print("Could not calculate key. Ensure both inputs contain letters.")

elif choice == 2:
    key = int(input("enter cipher key: "))
    S = input("enter a cipher text: ")

    plain_text = ""
    for char in S:
        if 'A' <= char <= 'Z':
            plain_text += chr((ord(char) - ord('A') - key) % 26 + ord('A'))
        elif 'a' <= char <= 'z':
            plain_text += chr((ord(char) - ord('a') - key) % 26 + ord('a'))
        else:
            plain_text += char

    print("Decrypted plain text:", plain_text)

elif choice == 3:
    key = int(input("enter cipher key: "))
    plain_input = input("enter a plain text: ")

    cipher_text = ""
    for char in plain_input:
        if 'A' <= char <= 'Z':
            cipher_text += chr((ord(char) - ord('A') + key) % 26 + ord('A'))
        elif 'a' <= char <= 'z':
            cipher_text += chr((ord(char) - ord('a') + key) % 26 + ord('a'))
        else:
            cipher_text += char

    print("Encrypted cipher text:", cipher_text)

else:
```

4---

```
elif choice == 2:
    key = int(input("enter cipher key: "))
    S = input("enter a cipher text: ")

    plain_text = ""
    for char in S:
        if 'A' <= char <= 'Z':
            plain_text += chr((ord(char) - ord('A') - key) % 26 + ord('A'))
        elif 'a' <= char <= 'z':
            plain_text += chr((ord(char) - ord('a') - key) % 26 + ord('a'))
        else:
            plain_text += char

    print("Decrypted plain text:", plain_text)

elif choice == 3:
    key = int(input("enter cipher key: "))
    plain_input = input("enter a plain text: ")

    cipher_text = ""
    for char in plain_input:
        if 'A' <= char <= 'Z':
            cipher_text += chr((ord(char) - ord('A') + key) % 26 + ord('A'))
        elif 'a' <= char <= 'z':
            cipher_text += chr((ord(char) - ord('a') + key) % 26 + ord('a'))
        else:
            cipher_text += char

    print("Encrypted cipher text:", cipher_text)

else:
    print("Invalid choice! Please select 1, 2, or 3.")

else:
    print("FUCK YOU GET HELL OUT FROM HERE.")
```

11. Testing Approach

Test Case	Input	Expected Result
Encryption	CAT, key 3	FDW
Decryption	FDW, key 3	CAT
Key finding	Plain: CAT, Cipher: FDW	Key = 3
Lowercase	cat, key 3	fdw
Spaces/punctuation	CAT! key 3	FDW! (punctuation preserved)
Invalid menu	Choice other than 1–3	Invalid choice message

12. Challenges Faced

- Handling alphabet wrap-around, such as shifting Z forward.
- Keeping uppercase and lowercase behavior consistent.
- Preserving spaces and punctuation instead of transforming them.
- Finding a key from plaintext and ciphertext while ignoring non-letter positions.
- Designing one menu that supports three different input/output flows.
- Avoiding incorrect results when the menu choice is invalid.

The use of modulo 26 is the main technique that keeps alphabetic shifts within the 26-letter alphabet.

13. Learnings & Key Takeaways

- Learned how a classical substitution cipher works.
- Practiced Python **input()**, conditional statements, loops, and string processing.
- Understood how **ord()** and **chr()** can manipulate characters.
- Used modulo arithmetic to implement circular alphabet shifts.
- Learned to separate encryption, decryption, and key-finding logic.
- Understood why non-letter characters should remain unchanged for readable output.

Key takeaway: a small amount of Python logic can model the complete encryption/decryption workflow of a classical cipher.

14. Future Enhancements

- Add a graphical user interface (GUI) instead of only console input.
- Allow the user to process text from a file.
- Add stronger input validation for keys and menu values.
- Provide an option to try all 26 possible shifts for educational cryptanalysis.
- Separate cipher functions into reusable Python functions/modules.
- Add automated unit tests for encryption, decryption, and key finding.
- Add modern cryptographic algorithms if the goal changes from education to real security.

These improvements would make the project easier to use, test, extend, and maintain.

15. References & Project Summary

References

- Python documentation and standard language features: `input()`, `print()`, loops, conditionals, strings, `ord()`, `chr()`.
- Classical Shift / Caesar Cipher concept and alphabet-shift method.
- The supplied project source code and project instructions provided for this report.

Project Summary

This Python project demonstrates a complete educational Shift Cipher workflow: understanding plaintext/ciphertext, selecting a key, encrypting, decrypting, and deriving the key from a known plaintext/ciphertext pair.

THANK YOU